

Kubernetes

Orchestrating Containers

Our Project

1. Managed with git
2. Running Docker containers
3. **Orchestrated with Kubernetes**
4. Tested in Gitlab
5. Deployed with ArgoCD
6. Monitored with Prometheus and Grafana

Orchestrating Containers on a Cluster

Situation: Have many containers and several machines

Goal: Get all of the containers running somewhere

- Making sure they are actually running
- Restarting them if they crash
- Facilitating communication between them
- Exposing external services to the world

Kubernetes (K8s)

Container orchestration in a cluster

Various **kinds** of resources declared in YAML files

→ Kubernetes tries to update cluster to match desired state

Grew out of internal Google project ("Borg")

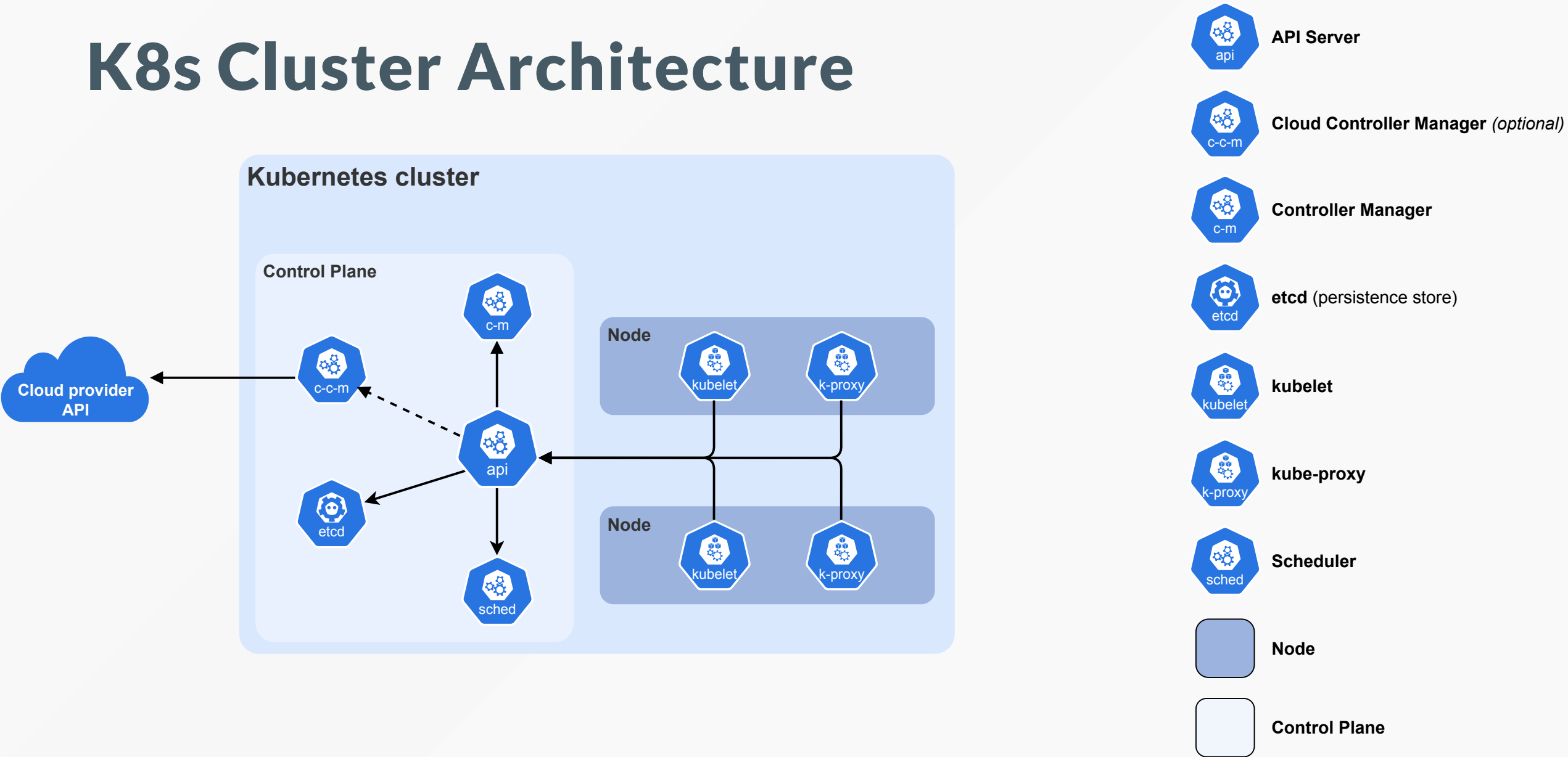
⇒ Many features and capabilities

⇒ Most teams won't need most features

Understanding Kubernetes

- Kubernetes is a huge project—we can't cover it all
- **Goals:**
 - Understand basic concepts of K8s
 - Gain familiarity with the K8s CLI
 - Deploy a few common elements on a K8s cluster
- **Non-goals:**
 - Knowing all K8s kinds
 - Knowing all configuration options
- **You know enough to learn what you need when you need it**

K8s Cluster Architecture



Minikube

Implementation of Kubernetes designed to run on a single node

Runs entirely within a VM or docker container

Often used for local development and testing

Activity: Start Minikube

Start Minikube with:

```
minikube start
```

This will take a few minutes

Once it's up, you should see both a client and server version from:

```
kubectl version
```

YAML Configuration in K8s

- Superset of JSON, designed to be human-readable
- Full language is (overly) complex

Atomic Types		Composite Types	
number	<code>12.345</code>	sequence	<code>[1, 2, 3, 4, 5]</code>
string	<code>"hello"</code>	mapping	<code>{"key": "value"}</code>
boolean	<code>true</code> and <code>false</code>		
null	<code>null</code>		

YAML Block Syntax

Alternative syntax for sequences and mappings relies on whitespace

Block Syntax

```
# YAML supports comments
key: "value"
nested:
  subkey: value
  list:
    - "Item 1"
    - "Item 2"
```

String quoting optional, unless it causes ambiguity

Flow Syntax (JSON)

```
{
  "key": "value",
  "nested": {
    "subkey": "value",
    "list": [
      "Item 1",
      "Item 2"
    ]
  }
}
```

K8s Pods: Smallest Unit of Compute

- One or more docker containers, always deployed together
 - Shared network namespace (access others on localhost)
 - Shared storage volumes
- Generally one main container
 - May have a **sidecar**: auxiliary container for logs, proxy, *etc.*
- Created and destroyed as needed
 - *Livestock, not pets*

Pod Definition

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web-service
spec:
  containers:
    - name: web-container
      image: web-service:latest
      imagePullPolicy: Never
      ports:
        - containerPort: 8000
          name: "http"
```

Create a new file

`project/services/pod.yaml` with these contents

Pod Definition: **kind**

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web-service
spec:
  containers:
    - name: web-container
      image: web-service:latest
      imagePullPolicy: Never
      ports:
        - containerPort: 8000
          name: "http"
```

The *kind* specifies the type of resource to be created

Pod Definition: `apiVersion`

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web-service
spec:
  containers:
    - name: web-container
      image: web-service:latest
      imagePullPolicy: Never
      ports:
        - containerPort: 8000
          name: "http"
```

Kubernetes kinds are versioned

New versions can be released without breaking old deploys

Mostly see in *beta* versions of new kinds

Pod Definition: Identifiers

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web-service
spec:
  containers:
    - name: web-container
      image: web-service:latest
      imagePullPolicy: Never
      ports:
        - containerPort: 8000
          name: "http"
```

`metadata.name`: Identifier for the pod

`metadata.labels.app`: Associates a pod with a **service** (We'll see this later)

`spec.containers[*].name`: Identifies a container within a pod (required, but superfluous for one-container pod)

Pod Definition: Container Image

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web-service
spec:
  containers:
    - name: web-container
      image: web-service:latest
      imagePullPolicy: Never
      ports:
        - containerPort: 8000
          name: "http"
```

Container image on Docker Hub
or other registry

To use a local image in minikube,
run:

```
minikube image load web-service:latest
```

`imagePullPolicy: Never`: Don't
even try to pull the image from
Docker Hub

Pod Definition: Network Ports

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web-service
spec:
  containers:
    - name: web-container
      image: web-service:latest
      imagePullPolicy: Never
      ports:
        - containerPort: 8000
          name: "http"
```

Specify which ports should be open for connections

`name` can be used on other objects to reference the port

Launching a Pod

From the `project/services` directory, run:

```
kubectl apply -f pod.yaml
```

Check that it has come up with:

```
kubectl get pods
```

Can add pod name to limit to specific pod

Note: `kubectl get` and similar commands will work with all K8s *kinds*

Examining a Pod

Add `-o yaml` to *get* to see a detailed configuration:

```
kubectl get pod -o yaml web-pod
```

The *describe* command gives detailed information about the state:

```
kubectl describe pod web-pod
```

The *logs* command show application logs:

```
kubectl logs web-pod
```

Debugging Technique: `exec` into Pod

The `exec` command will run a process inside an existing pod

To get a terminal in our *web-pod*:

```
kubectl exec -it web-pod -- /bin/sh
```

Similar options to `docker exec`

The `--` separates the command (and its options) from options to `kubectl exec`

Connecting to a Server in a Pod

With a shell in a pod, we can access the server on localhost:

```
wget -O - http://127.0.0.1:8000/api/visits # From within web-pod ONLY
```

Port Forwarding to a Pod

Forward this port to your local machine (where `kubectl` is running):

```
kubectl port-forward pod/web-pod http
```

`http` is `ports[*].name` in configuration

→ Can use port numbers instead

→ Uses same port locally by default; change with `3000:http`, e.g.

Now, connect to local machine:

```
curl http://localhost:8000/api/visits # On development machine
```

Deleting a Pod

Pod can be deleted with:

```
kubectl delete pod web-pod
```

... and created anew with:

```
kubectl apply -f pod.yaml
```

Note that this creates a new container
⇒ Visit count gets reset

Persistent Volumes

Persistent Volumes are storage available on the cluster

- Created by cluster administrator
- An instance of a **Storage Class**
 - Local to cluster, or a cloud service
 - Each cloud provider has its own options

Applications request a persistent volume with a **Persistent Volume Claim**

Persistent Volume Claim (PVC)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-pvc
spec:
  storageClassName: standard
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Mi
```

Create a new file:

```
project/services/pvc.yaml
```

`metadata.name` is arbitrary

Persistent Volume Claim: Storage Class

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-pvc
spec:
  storageClassName: standard
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Mi
```

`spec.storageClassName` must be available on the cluster

To see available classes:

```
kubectl get storageclass
```

default class used if not specified

Persistent Volume Claim: Access Modes

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-pvc
spec:
  storageClassName: standard
  accessModes:
    - ReadWriteOnce
resources:
  requests:
    storage: 10Mi
```

- **ReadWriteOnce**
 - Connected to single node with read/write capability
 - Supported by most classes
- **ReadOnlyMany**
 - Multiple nodes, read-only
 - Many classes support
- **ReadWriteMany**
 - Only select classes

Persistent Volume Claim: Resource Request

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-pvc
spec:
  storageClassName: standard
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Mi
```

Amount of storage available on the volume

Creating a Persistent Volume

```
kubectl apply -f pvc.yaml
```

See the claim:

```
kubectl get pvc
```

See the volume that got created:

```
kubectl get pv
```

Mounting the Volume on the Pod

```
...
spec:
  containers:
    - name: web-container
      ...
      volumeMounts:
        - mountPath: /var/lib/web-service/
          name: visits-volume
  volumes:
    - name: visits-volume
      persistentVolumeClaim:
        claimName: web-pvc
```

Edit

```
project/services/pod.yaml
```

Add these lines to the bottom of the file

Mounting the Volume on the Pod

```
...  
spec:  
  containers:  
    - name: web-container  
      ...  
      volumeMounts:  
        - mountPath: /var/lib/web-service/  
          name: visits-volume  
  volumes:  
    - name: visits-volume  
      persistentVolumeClaim:  
        claimName: web-pvc
```

Reference the volume associated with the claim we just made

Mounting the Volume on the Pod

```
...
spec:
  containers:
    - name: web-container
      ...
      volumeMounts:
        - mountPath: /var/lib/web-service/
          name: visits-volume
  volumes:
    - name: visits-volume
      persistentVolumeClaim:
        claimName: web-pvc
```

These names must match

Mounting the Volume on the Pod

```
...
spec:
  containers:
    - name: web-container
      ...
      volumeMounts:
        - mountPath: /var/lib/web-service/
          name: visits-volume
  volumes:
    - name: visits-volume
      persistentVolumeClaim:
        claimName: web-pvc
```

Mount point for this volume
in container

Multiple Documents in One File

Multiple YAML documents can be stored in the same file

Separate with `---` on its own line

`kubectl apply -f` will process all documents in a file

⇒ Keep related components in the same file

Activity: Pod and PVC Together

1. Create a new YAML file in `project/services`
2. Copy in both the contents from `pod.yaml` and `pvc.yaml`, separated by a line containing `---`
3. Use `kubectl apply -f` on this file
4. What do you notice happens?

Kubernetes Workloads

We shouldn't be managing pods directly (*Livestock, not pets*)

We should describe what's required, and let K8s create the pods

- **Deployment:** Ensure a given number of a pod are running
 - Deployment pods are interchangeable
 - Good for stateless applications (e.g. web servers)
- **StatefulSet:** Identical pods, but distinct identities
 - Persistent volumes can be attached to an identity
 - Good for stateful applications (e.g. databases)

Additional K8s Workloads

- **ReplicaSet:** Used under the hood by Deployments
- **DaemonSet:** Ensures one pod per node
 - Often used for cluster services (e.g. networking)
- **Job:** Pods that run a task to completion, then exit
- **CronJob:** Run a Job periodically

Creating a Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  namespace: dso-project
  labels:
    app: web-service
spec:
  selector:
    matchLabels:
      app: web-service
  replicas: 1
  template:
```

Create a new file

```
project/services/web-
service.yaml
```

with this contents

Specifying the pods

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  namespace: default
  labels:
    app: web-service
spec:
  selector:
    matchLabels:
      app: web-service
  replicas: 1
  template:
```

The Deployment will make `replica` copies of the pod described in `template`

Paste the `metadata` and `spec` mappings from the Pod definition below. Indent it twice.

Remove `metadata.name` and the persistent volume configuration

Specifying the pods

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  namespace: dso-project
  labels:
    app: web-service
spec:
  selector:
    matchLabels:
      app: web-service
  replicas: 1
  template:
```

```
# File continues...
  metadata:
    name: web-pod
    labels:
      app: web-service
  spec:
    containers:
      - name: web-container
        image: web-service:latest
        imagePullPolicy: Never
        ports:
          - containerPort: 8000
            name: "http"
```

Namespaces

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  namespace: dso-project
labels:
  app: web-service
spec:
  selector:
    matchLabels:
      app: web-service
  replicas: 1
  template:
```

Namespaces organize resources in the the cluster

- See just those components relevant to a project
- Avoid name collisions across projects
- Restrict access with RBAC
- Limit resources per project

Investigating Namespaces

Show all namespaces:

```
kubect1 get namespaces
```

Get pods within *my-namespace*

```
kubect1 get pods -n my-namespace
```

Get pods in all namespaces:

```
kubect1 get pods --all-namespaces
```

Activity: Investigating Namespaces

1. What namespaces are currently on the cluster?
2. What pods are running in the *kube-system* namespace?

Adding a Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: dso-project
---
```

Add this Namespace definition to the beginning of the `web-service.yaml` file

Deploy the Deployment

```
kubectl apply -f web-service.yaml
```

See if a pod got created

```
kubectl get pods -n dso-project
```

Or, switch the current namespace for `kubectl`:

```
kubectl config set-context --current --namespace=dso-project  
kubectl get pods
```

Deployment Ensures Pods Exist

Delete the current pod:

```
kubect1 delete pod web-deployment-7447478ddd-j9fv4 # Name will vary
```

See that a new pod gets created:

```
kubect1 get pods
```

Services: Abstracted Applications

Pods maybe be moved across nodes, change IP address, and get new names (for deployments). How can we reliably access an application?

A **Service** provides a stable IP and DNS name, routing all traffic to the pods actually running the application.

Creating a Service

```
---
apiVersion: v1
kind: Service
metadata:
  name: web-server
  namespace: dso-project
spec:
  type: ClusterIP
  selector:
    app: web-service
  ports:
    - name: http
      port: 8000
      targetPort: http
```

Add another YAML document to
`web-service.yaml`

Creating a Service: Routing to the Pods

```
---
apiVersion: v1
kind: Service
metadata:
  name: web-server
  namespace: dso-project
spec:
  type: ClusterIP
  selector:
    app: web-service
  ports:
    - name: http
      port: 8000
      targetPort: http
```

The `selector` indicates which pods should be routed to as part of this service

`targetPort` indicates the port on that pod to receive the traffic

Creating a Service

```
---
apiVersion: v1
kind: Service
metadata:
  name: web-server
  namespace: dso-project
spec:
  type: ClusterIP
  selector:
    app: web-service
  ports:
    - name: http
      port: 8000
      targetPort: http
```

- **ClusterIP:** Service accessible only inside cluster (the default)
- **NodePort:** Opens a port on *every* node in the cluster
 - All traffic to that port routes to service
- **LoadBalancer:** Provisions a load balancer routing service
 - Requires provider support

Deploy the Service

```
kubectl apply -f web-service.yaml
```

See the service:

```
kubectl get service # or svc
```

Forward a local port to the service:

```
kubectl port-forward service/web-server http
```

Activity: Changing the Number of Replicas

1. Increase the number of replicas in the deployment YAML
2. Deploy the updated YAML
3. Check that the expected number of pods have been created
4. Try accessing the service through port-forwarding

Serving with a LoadBalancer

`kubectl port-forward` always routes to the same pod for a service

Minikube can simulate a LoadBalancer:

1. In a separate terminal, run `minikube tunnel`
2. Change the service's `spec.type` to "LoadBalancer"
3. Re-deploy the service
4. Check the service with `kubectl get svc`
5. User the EXTERNAL-IP of the service to access it:

```
curl http://10.110.165.143:8000/api/visits
```

Other Ways to Expose Applications

- An **Ingress** routes HTTP(S) traffic to services based on path
 - Allows one load balancer to serve many services
 - Can do SSL/TLS termination
- The **Gateway API** is a new system for managing K8s networking
 - The **Gateway Class** defines the infrastructure
 - The **Gateway** specifies ports, protocols, and TLS termination
 - An **HTTPRoute** maps HTTP traffic to the appropriate service

Activity: Create an Ollama Service

Ollama is an inference server: It can run Large Language Models (LLMs) to generate text

In `exercises/ollama-service/ollama-pod.yaml`, there is configuration for a pod that uses Ollama to run the *qwen3:0.6b* model

Create a Deployment and a Service to run Ollama in the cluster
→ Use the existing *dso-project* namespace

See `exercises/ollama-service/README.md` for more details

Activity: Create an Ollama Service

A sample solution can be found in `reference/services/ollama.yaml`

Make sure configuration code for the Ollama service is stored in `project/services/`, either from your work or the reference

Access Ollama from Another Pod

Our *web-service* offers a UI for sending queries to a LLM

⇒ It needs to send queries to the *ollama* Service

All Services are given names in the cluster DNS. The service *my-service* running in *my-namespace* may be reached at:

- `my-service`, from pods in *my-namespace*
- `my-service.my-namespace`, from all pods
- `my-service.my-namespace.svc.cluster.local`, for a FQDN

Getting a Debugging Pod

Often useful to create a pod and get shell to debug network issues:

```
kubectl run debug-pod -it --rm --image=nicolaka/netshoot --restart=Never -- sh
```

- `debug-pod` is pod name; is arbitrary
- `nicolaka/netshoot` has several network troubleshooting tools
 - Also, `busybox` is very lightweight Linux shell
- `-- sh`: Run the *sh* shell
 - Alternatively, `bash` is often available

Setting Environmental Variables

The *web-service* application uses the `OLLAMA_URL` environmental variable to find the Ollama server

Add to the *web-service* Deployment:

```
spec:
  template:
    spec:
      containers:
        - name: web-service
          env:
            - name: OLLAMA_URL
              value: "http://ollama-service.dso-project.svc.cluster.local:11434"
```

Activity: Test the Web Site

1. Deploy the updated deployment.
2. Set up port-forwarding for the service. (Codespaces doesn't make accessing a minikube tunnel address easy.)
3. Visit the site in your browser, and try asking a question.

Persisting the Visit Counter

- Currently, each pod keeps its own visit count
 - Count gets reset when pod deleted and recreated
 - Count depends on which pod you reach
- Could move to StatefulSet; give each pod a Persistent Volume
 - Would still need a consensus algorithm to merge counts
- Better approach: Separate persistence service
 - **A database**

Adding an Object Store

- Our needs are minimal $\Rightarrow$ Almost any database would work
- We choose to use an S3-compatible object store
 - Store and retrieve files at a given key
 - Plenty of tooling for Amazon's S3 to reuse
 - Our Python server accepts S3 URLs for the counter file
- We have selected [Garage](#) as our object store

Helm Configures a Configuration

- Projects such as Garage require many resources to be configured
- We could write a bunch of YAML documents, but ...
 - Most deploys will look about the same
 - We need to make sure names are consistent across resources
- **Helm** is a package manager for Kubernetes
 - Works from a **chart**, a set of templated configuration files
 - Customized values can be inserted during deployment

Helm Charts

- Helm charts can be:
 - Read from a Helm **repository**
 - A local `.tgz` file with the appropriate structure
 - A local directory with the appropriate structure
- We will use a Helm chart from [datahub-local/garage-helm](https://datahub-local.github.io/garage-helm)

Getting the Chart

Add the repo with this chart to our Helm configuration:

```
helm repo add garage-helm https://datahub-local.github.io/garage-helm
```

See all of the configurable values for the `garage-helm/garage` chart:

```
helm show values garage-helm/garage
```

Configuring Our Installation

```
deployment:
  replicaCount: 1

garage:
  replicationFactor: "1"

persistence:
  enabled: true
  data:
    storageClass: standard
    size: 1Gi
  meta:
    storageClass: standard
    size: 100Mi
```

```
clusterConfig:
  enabled: true
  buckets:
    - name: visitors
  keys:
    my-key:
      keyId: GK012...4567
      secretKey: 0123...abcdef
      buckets:
        - visitors
```

Full file at

`reference/helm/garage.yaml`

Installing the Chart

Install the chart with our configuration:

```
helm install garage garage-helm/garage --namespace garage \
  --create-namespace --values garage.yaml
```

This will take a few minutes to run

Once finished, check for services and pods:

```
kubectl get service -n garage
kubectl get pods -n garage
```

Breaking Down the Install Command

```
helm install garage \    # This is the name of the installation. It is used  
                        # both in helm itself and within k8s resources.  
  
    garage-helm/garage \ # The name of the helm chart in the helm repo.  
  
    --namespace garage \ # The k8s namespace where we will deploy this.  
  
    --create-namespace \ # Create that namespace during installation.  
  
    --values garage.yaml # Load values from this file.
```

Activity: Make *web-service* Use Garage

The *web-service* can be configured to use Garage by setting these environmental variables in the Pods:

`AWS_ENDPOINT_URL`: *http://garage.garage.svc.cluster.local:3900*

`AWS_ACCESS_KEY_ID`: The `keyId` in the Garage configuration file

`AWS_SECRET_ACCESS_KEY`: The `secretKey` in the Garage configuration

`VISIT_COUNTER_FILE`: *s3://visitors/visit.txt*

Deploy the updated service. Now the visit counter should persist even when the underlying pods are restarted.

Other Kubernetes Topics

ConfigMaps

- **ConfigMaps** store key-value pairs separately from any resource
 - Separate "dynamic" configuration from "static" values
 - Share configuration options across several resources
- Values can be used
 - In a container launch command
 - As values for environmental variables
 - Mounted as files inside containers

Secrets

- **Secrets** are similar to ConfigMaps, but intended for sensitive data
 - *e.g.* Passwords, tokens, keys
- Keep such information out of pod images and specs
- Any user with ability to create a pod can access Secrets
 - Use RBAC to restrict access to Secrets
 - Restrict access to Secret to necessary containers

Role-Based Access Control (RBAC)

- **Roles** describe certain capabilities: what actions are allowed on which resources
- **RoleBindings** grant certain roles to certain **Users**
- Users may be:
 - Human users, managed outside of Kubernetes
 - Service Accounts, allowing pods to communicate with the K8s API server

Kubernetes Dashboard

The currently-recommended dashboard is [Headlamp](#), which can run as an local desktop application, or on the cluster itself

I'd recommend running locally for production

```
# Minikube-specific instructions
minikube addons enable headlamp
minikube addons enable metrics-server

kubectl create token headlamp -n headlamp # Copy this value

kubectl port-forward -n headlamp service/headlamp 8080:80 --address 0.0.0.0
```

Use token to authenticate with UI

Network Policies

By default, all pods can communicate with each other

Adding a **NetworkPolicy** switches this to *default-deny*
→ Only explicitly allowed traffic is permitted

Rules based on pod labels, namespace labels, or IP blocks

E.g. The *web-service* should only have

- Incoming traffic on port 8000
- Outgoing traffic to DNS and *ollama*

Resource Requests and Limits

- Resource **requests** and **limits** govern how much CPU and memory a pod may consume
 - CPU measured in *cores* or *millicores* (`250m` = $\frac{1}{4}$ core)
 - Memory in bytes, usually with suffixes (`Mi` = 2^{20} , `Gi` = 2^{30})
- Requests are used when scheduling pods on nodes
 - The total requested by all pods cannot exceed total available
- Limits enforced at runtime
 - CPU usage throttled
 - Excess memory usage results in OOM kill

Matching Pods to Nodes

The **kube-scheduler** decides which node hosts which pods

Modern AI workloads need certain pods running on specific hardware

- A **node selector** on a pod specifies labels that must be present on the node to which it is assigned
- A **node affinity** on a pod expresses a preference about where the pod should be scheduled
- A **taint** on a node prevents pods from being scheduled there if they don't have a matching **toleration**

Container Probes

- **Probes** are used to check on container status
 - **startupProbe** checks if a container has finished starting
 - Other probes disabled until it succeeds
 - Stops checking after success
 - **readinessProbe** checks if container is ready for traffic
 - Services only send traffic to ready containers
 - **livenessProbe** checks if container is still working
 - Container will be killed and restarted on failure

Feedback



form.typeform.com/to/RkXh2Poja